

ksat / sat-solvers-kotlin

Runtime benchmarks: C vs Kotlin/JVM vs Kotlin/Native

Measured, not estimated

Contents

- What is measured 3
- Results 3
- Why is Kotlin/Native slower than the JVM here? 3
- Reproduce 4

What is measured

Solve time on pigeonhole instances $\text{PHP}(n+1, n)$: $n+1$ pigeons into n holes, which is UNSAT and forces a CDCL solver through (super-)exponential search. These are ideal for a runtime comparison because the work is pure search — no I/O bias, deterministic, and it scales sharply with n , so we can land an instance in the few-seconds range where actual solving (not process startup) dominates.

Instances are generated by `shadow/tools/gen_cnf.py` (up to `php_8_7`) and `shadow/tools/gen_big_php.py` 8 9 (`php_9_8`, `php_10_9`).

C original MiniSat reference, `clang++ -O2 -std=c++11`, whole-process wall time (parse + solve + exit; parse/IO is negligible vs solve on these instances).

Kotlin/JVM `:minisat:runBenchmark, solve()` only, median of 5 runs after 3 warm-ups (the JIT must warm up before numbers mean anything).

Kotlin/Native the `macOS arm64 release (-opt)` executable `(:minisat:linkReleaseExecutableMacosArm64)`, `solve()` only, same median-of-5 after-3-warm-up protocol, timed with `TimeSource.Monotonic`.

Machine: Apple Silicon (arm64), macOS. All three run the SAME MiniSat algorithm (the C reference and the shared Kotlin port in `:minisat`).

Results

instance	C	Kotlin/JVM	Kotlin/Native	JVM / C	Native / C
php_9_8	0.180 s	0.239 s	0.650 s	1.33×	3.61×
php_10_9	2.606 s	3.526 s	9.641 s	1.35×	3.70×

The ratios are stable across both instances:

- **Kotlin/JVM $\approx 1.3\times$ C** — a small constant factor; the port is competitive.
- **Kotlin/Native $\approx 3.7\times$ C** — and, notably, **slower than Kotlin/JVM** ($\approx 2.7\times$ the JVM time).

Why is Kotlin/Native slower than the JVM here?

This surprises people (“native should be faster”), but it is a known effect for this kind of workload. “Native” means **no VM at runtime**, not **as fast as C**.

1. **JIT beats AOT on long-running hot loops.** The JVM is not interpreting here: HotSpot profiles the `solve()` hot loop at runtime and recompiles it with optimizations a static compiler cannot safely apply — profile-guided inlining along the paths actually taken, bounds-check elimination on the hot arrays, devirtualization of monomorphic calls, loop unrolling. For a solve that runs for seconds, the compile cost is fully amortized. Kotlin/Native is **ahead-of-time** (LLVM) and must stay conservative — it has no runtime profile.
2. **The workload is allocation- and array-heavy.** CDCL solvers hammer arrays, watchlists, and clause allocation. What matters is how well the compiler eliminates bounds checks, does escape analysis (stack- vs heap-allocation), and how the GC behaves. The C reference has **no bounds checks and no GC at all**; the JVM claws a lot of that back with a mature JIT + generational GC; Kotlin/Native (LLVM backend, newer GC) is currently less favourable for exactly this pattern.
3. **GC model.** JVM: a highly-tuned throughput GC. Kotlin/Native: a newer GC, less suited to this allocation storm. C: none.

So the ordering $C < \text{JVM} < \text{Native}$ (slower) is plausible: C has zero runtime overhead, the JVM recovers most of the gap via JIT + a mature GC, and Kotlin/Native pays an AOT penalty **without** C’s zero-overhead memory model.

Control: the release build really is optimized

To rule out a measurement artifact, the **debug** native binary (`linkDebugExecutableMacosArm64`) solves `php_9_8` in **7860 ms** — 12× slower than the **release** binary's 650 ms. So the 650 ms figure is genuinely the `-opt` code, and the 3.6× gap to C is real, not an un-optimized build.

Caution

These are **JVM and macOS-arm64-native** numbers on one machine. They are not a claim about every platform or every instance family — but the JVM/C and Native/C ratios are each stable across the two PHP sizes measured. No perf statement is made without these measured numbers (see also `shadow/BENCHMARK_RESULTS.md` for the raw table).

For the game (`luckySweeper`) this matters: it runs the Kotlin MiniSat on the **JVM** (Android/desktop), i.e. the 1.3×-of-C column — the fast one. Kotlin/Native would be the relevant column only for an iOS/native build of the solver.

Reproduce

```
# C reference
clang++ -O2 -std=c++11 -o shadow/minisat-c/minisat_bench shadow/minisat-c/
minisat_trace.cc
time shadow/minisat-c/minisat_bench shadow/cnf/php_10_9.cnf

# Kotlin/JVM
./gradlew :minisat:runBenchmark --args="$(pwd)/shadow/cnf/php_10_9.cnf"

# Kotlin/Native (macOS arm64, release)
./gradlew :minisat:linkReleaseExecutableMacosArm64
minisat/build/bin/macosArm64/releaseExecutable/minisat.kexe \
shadow/cnf/php_9_8.cnf shadow/cnf/php_10_9.cnf
```